



Draw a Square with a Loop

Ontario Math C3.1 — repeating events

Name: _____

Date: _____

★ I can use a for-loop to make Bit draw a square.

Four sides, one loop

A square has 4 equal sides and 4 square corners (right 90). Instead of typing forward and turn four times, a loop does it for you.

Read the code: `range(4)` repeats forward 50, right 90 — four times — and Bit draws a square.

Read this Python code

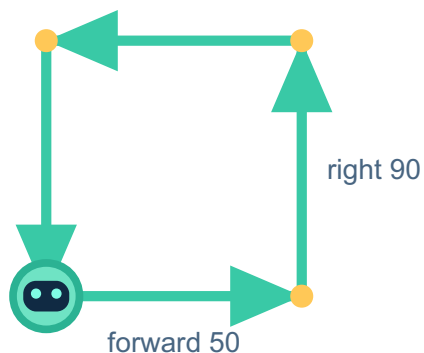
```
for i in range(4):  
    bit.forward(50)  
    bit.right(90)
```

PYTHON

OUTPUT

Bit draws a square and ends back at the start.

`range(4)` → a square



`range(4)` of forward 50, right 90 → a square.

1 Read it

How many sides does Bit draw? _____ How many right-90 turns? _____

2 Predict, then run

You change `forward(50)` to `forward(100)` (`range(4)` and `right(90)` stay). PREDICT: is it still a square? What changes — the shape or the size? Then run it.

3 Write your own

Finish the code so Bit draws a square with sides of 60. Fill in the two numbers: `for i in range(_____):`
`bit.forward(60) bit.right(_____)`

4 Challenge

Bit draws a square with `forward(50)`, four times. How long is the whole path Bit draws all the way around? (This is the square's perimeter.)

Big idea

A square = repeat 4 of (forward, right 90). The forward number sets the SIZE; `range(4)` and `right 90` keep it a SQUARE.



Draw a Square with a Loop

Quick facilitation notes & answer key

What this worksheet teaches

A square has 4 equal sides. Instead of typing "go forward, then turn" four times, the student uses a shortcut that means "repeat this 4 times." They read the shortcut, change a number to see what happens, fill in a missing piece, and add up the distance all the way around (the perimeter). No coding experience needed — everything you need is on the page and in the answer key.

Before you start — showing the drawing (optional)

The worksheet shows the instructions AND a picture of the finished square, so you can teach straight from the printout — no computer required.

To show it live (optional): open a browser, go to a free "Python turtle" playground (e.g. search "trinket python turtle"), type the few lines from the worksheet, and press Run — a turtle draws the square. You only press Run.

How to lead it (about 30 minutes)

1. Remind them: last lesson "repeat 3 times" drew 3 sides; one more side closes a square.
2. Show it (you do). Read the code like a recipe: "Repeat 4 times: forward 50, then turn right." Note Bit ends back where it started.
3. Let them work. Students do Exercises 1–4, solo or in pairs. Ex 4 (challenge) adds up all four sides (the perimeter).
4. Wrap up: the "forward" number sets how BIG the square is; "repeat 4 times" and the right turn keep it a SQUARE.

Answer key (these are the verified correct answers)

Exercise 1 — 4 sides and 4 turns.

Exercise 2 — still a square; only the SIZE gets bigger, not the shape (a side of 100 makes the distance around 400).

Exercise 3 — the missing numbers are 4 (repeat 4 times) and 90 (turn right 90).

Exercise 4 (challenge) — the whole path is $4 \times 50 = 200$, the square's perimeter.

If students get stuck — and ways to adjust

Watch for: changing the "repeat 4 times" number in Ex 2 when only the side length should change — a square always has 4 sides, so that 4 stays the same.

Make it easier: trace one side and count 50, 100, 150, 200 around the square for the perimeter.

Make it harder: ask for the perimeter of a square with sides of 70 (280).

Ontario curriculum link (official wording)

C3.1 — solve problems and create computational representations of mathematical situations by writing and executing code, including code that involves sequential, concurrent, and repeating events.

In plain terms: students write and run step-by-step instructions — including a "repeat" instruction — to make something happen.

C3.2 — read and alter existing code, including code that involves sequential, concurrent, and repeating events, and describe how changes to the code affect the outcomes.

In plain terms: students read instructions, change one, and explain what the change did to the result.

You'll know it worked when

The student reads the repeat loop (Ex 1), explains size-vs-shape when changing the side length (Ex 2), fills in the missing numbers (Ex 3), and finds the perimeter (Ex 4).